Escape the Geese – Method Selection and Planning

Cohort 2 Team 3 – ‘Kostas Barmpis and Dimitris Kolovos’

Team members:

Sruthiksha Ananth

Louis Price

Sarah James

Ben Smejko

Henrik Bowden

Tim Giles

June Njiru

Software Engineering Method

We chose Agile for our method because it has short and iterative development cycles. This allows for the requirements to change as we work. Instead of finalising all requirements at the beginning, this allows us to review and refine our work based on customer feedback and our progress.
This was suitable for this project because we were working in a small team with a fixed deadline to deliver a playable game. We applied this approach through the requirements, architecture and implementation stages. For example, our requirements table was revised after talking to the customer.
Alternative methods considered were waterfall and Scrum-lite weekly sprints. Since we made revisions to the breakdown of our implementation plan by reviewing and refining our architecture, it would have been more difficult to do if we were using the Waterfall method as it requires everything to be planned upfront. The Agile method was also chosen as it allows for our working software to be delivered early so we can exchange our prototype with the second team efficiently. Problems are also discovered earlier so they can be fixed gradually without having to deal with problems right before the fixed deadline. We decided against Scrum-lite with weekly sprints due to our small team size. Instead, we used more flexible weekly planning, allowing us to adjust tasks and responsibilities based on our progress while still working towards the final deadline.

Development and Collaboration tools

The following tools were selected to support the development, documentation and organisation of the project.

Purpose	Tool	Justification
Version Control and Task tracking	GitHub (repository, Issues, Milestones, Projects board)	Keeps code , task tracking and the website in one place. Branches and pull requests let several people work at once without overwriting each other, and the board's history provides evidence of planning.
Game framework and build	LibGDX with Gradle (Java 17), generated with gdx-liftoff	Meets the Java requirements (CR_JAVA) and is a stable, well-documented 2D framework, as the customer asked. Gradle handles dependencies and builds one executable JAR that runs on Windows, macOS and Linux (CR_JAR, CR_PLATFORM)
IDE	IntelliJ IDEA	Free for students, with built-in Gradle and Git support and good java debugging and refactoring tools. Using the same IDE helps team members help each other.

Diagrams	PlantUML	Diagrams are written as text, so they can be version-controlled in Github alongside the code, and changes are easy to trace for the architecture and evolution section.
Communication	WhatsApp, plus in-person meetings	Quick day-to-day contact that everyone already uses, with weekly in person meetings for planning and decisions.

Team Organisation

Responsibilities were distrubuted among the team member in the different areas of the project, assigning each member ownership of features, risks and other tasks to ensure the monitoring of progress. Although individuals were responsible for specific parts of the project, decisions were discussed and agreed upon by the team with members taking the lead as necessary. Tasks were divided among team members through a combination of volunteering and assignment. We tracked these using GitHub Issues and the project board, allowing us to keep track of what needed to be done, who was responsible and the progress being made.
We also held meetings weekly, on Mondays and Tuesdays. Monday meetings focused on reviewing what had already been done and agreeing on the general directionn forward, while Tuesday meetings were lab sessions where we could all come together and work on the project. Between meetings, we communicated through Whatsapp and email to share updates, ask questions and coordinate the work. This approach suited our team because it allowed members to take responsibility for their assigned work while still collaborating on decisions and working towards the same deadline.

Project Planning

Our initial Gantt chart outlined the main stages of the project and when we expected to begin each stage, including requirements gathering, architecture design and implementation and a rough breakdown of the implementation tasks. At this point, the plan was still broad because we had not yet finalised our architecture or determined exactly how the implementation would be divided.

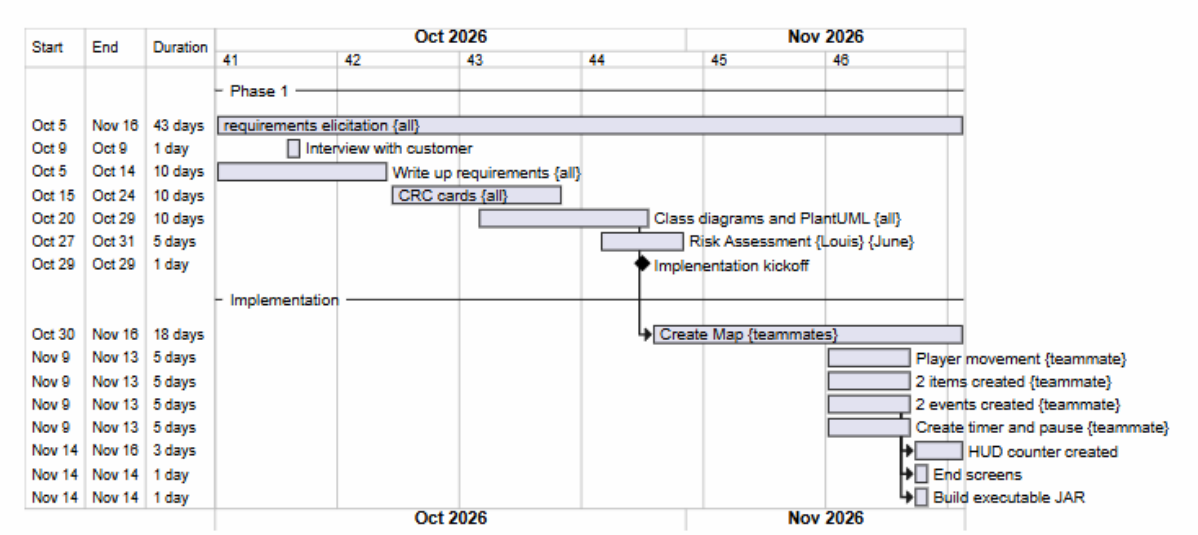


Figure 1: Gantt chart version

This Gantt chart detailed what we wanted to do from 5th October to 27th October. As the project progressed, we made changes to our initial plan, for example during the implementation stage. Once we had a clearer understanding of the architecture, we were able to break down the broader development goals into more specific tasks. Our weekly plans became more detailed, showing what needed to be completed within each week rather than only identifying the overall development phase.

.

ID	Workstream	Task	Output / evidence	Linked requirement or section	Priority
Planning & Method					
PM1	Planning & Method	Agree team roles (lead dev, requirements, architecture, planning/risk, website, customer liaison) and how work is allocated	Team organisation paragraph	Method Selection & Planning	High
PM2	Planning & Method	Set up GitHub milestones (Assessment 1, Assessment 2), priority/type labels and the project board	Board, milestones and labels live on GitHub	Method Selection & Planning	High
PM3	Planning & Method	Create a GitHub issue for every Assessment 1 task in this sheet, with labels and assignees	Issues linked to the A1 milestone	Method Selection & Planning	High
PM4	Planning & Method	Choose and justify the software engineering method (e.g. Agile, Scrum-lite with weekly sprints) and say why alternatives were rejected	Method section draft	Method Selection & Planning	High

Figure 2: Plan for October 27 - November 1st

Escape the Geese: Week Plan (Tue 27 Oct to Sun 1 Nov 2026)

Goal this week: draft Method Selection & Planning and Risk Assessment, fix the requirements document, and get a playable Assessment 1 build running. Assessment 1 deadline: M

Yellow cells (Owner, Status, Notes) are for the team to fill in. Status uses a dropdown; the Summary sh

ID	Workstream	Task	Output / evidence	Linked requirement or section	Priority	Owner	Due	Depends on
Planning & Method								
PM3	Planning & Method	Create a GitHub issue for every Assessment 1 task in this sheet, with labels and assignees	Issues linked to the A1 milestone	Method Selection & Planning	High		Thu 29 Oct	PM2
PM4	Planning & Method	Choose and justify the software engineering method and why alternatives were rejected	Method section draft	Method Selection & Planning	High		Fri 30 Oct	
PM5	Planning & Method	List development and collaboration tools with a one-line justification each	Tools table	Method Selection & Planning	Medium		Fri 30 Oct	
PM6	Planning & Method	Build the Gantt chart to 16 Nov with dependencies, priorities and the critical path	Gantt chart v1	Method Selection & Planning	High		Fri 30 Oct	PM3
Risk Assessment								
RA1	Risk Assessment	Write the risk management intro: process	Risk intro draft	Risk Assessment & Mitigation	High		Thu 29 Oct	
RA2	Risk Assessment	Draft the risk register	Risk register v1	Risk Assessment & Mitigation	High		Fri 30 Oct	RA1
RA3	Risk Assessment	Review the register at the team meeting	Risks page v1	Website / risk evidence	Medium		Sun 01 Nov	RA2
Requirements								
RQ1	Requirements	Fix requirements doc: 'Quantity' to 'Quality', UR_ACCESS to UR_ACCESSIBILITY, add Priority to FR/QR tables, QR_STARTUP to a couple of seconds	Requirements v2	Requirements	High		Wed 28 Oct	
RQ2	Requirements	Add the new IDs from customer meeting 1 (FR_TUTORIAL, FR_EXHAUSTION_METER, FR_ITEM_TYPES, FR_EVENT_VISUAL, etc.)	Requirements v2	Requirements	High		Thu 29 Oct	RQ1
RQ3	Requirements	Book customer meeting 2: confirm new requirements and user testing in the labs	Meeting booked; questions list	Requirements	Medium		Fri 30 Oct	

Figure 3: Plan For November 1st onwards:

RQ3	Requirements	Book customer meeting 2: confirm new requirements and user testing in the labs	Meeting booked; questions list	Requirements	Medium		Fri 30 Oct	
Implementation								
IM1	Implementation	Merge the code skeleton into the repo; check it builds and runs on Windows, macOS and Linux	Game opens on all three OSs	CR_PLATFORM, CR_JAVA	High		Wed 28 Oct	
IM2	Implementation	Simple map with clear boundaries the player cannot leave (A1 map can be very simple)	Map in game	FR_MAP_BOUNDS	High		Thu 29 Oct	IM1
IM3	Implementation	Player movement with WASD and arrow keys	Player moves	FR_MOVE_KEYS	High		Thu 29 Oct	IM1
IM4	Implementation	Geese arrive in waves and block the player's movement	Geese waves in game	FR_GEESE_WAVES, FR_GEESE_BLOCK	High		Fri 30 Oct	IM3
IM5	Implementation	Two initial inventory items that disperse or distract geese (A1 needs two)	2 working items	FR_START_ITEMS, FR_ITEM_DISPERSE	High		Sat 31 Oct	IM4
IM6	Implementation	Two hidden events, invisible until triggered, each with a sound and visual cue (A1 needs two)	2 working events	FR_EVENTS_COUNT, FR_EVENT_AUDIO, FR_EVENT_VISUAL	High		Sat 31 Oct	IM3
IM7	Implementation	5-minute timer; game starts paused and can be paused at any time	Timer and pause working	FR_TIMER, FR_START_PAUSED, FR_PAUSE	High		Fri 30 Oct	IM1
IM8	Implementation	On-screen counter of items used and events triggered (no scoring needed for A1)	HUD counter	FR_USAGE_COUNTER	High		Sat 31 Oct	IM5, IM6
IM9	Implementation	Win and loss screens	End screens	FR_WIN, FR_LOSE	Medium		Sun 01 Nov	IM7
IM10	Implementation	Start a licence log for every asset and library used	Licence log	CR_LICENCE	Medium		Sun 01 Nov	
IM11	Implementation	Build the executable JAR (lwjgl3:jar) and test it on a second computer	Working JAR	CR_JAR	Medium		Sun 01 Nov	IM9
Architecture								
AR1	Architecture	Check class names in the diagrams match the code; note any changes from the CRC cards for the evolution section	Change notes; updated diagrams	Architecture	Medium		Sun 01 Nov	IM1

Figure 4: We then added more tasks for implementation.

As we had finished with architecture our plan could evolve to have a more clear and detailed section for the implementation phase.

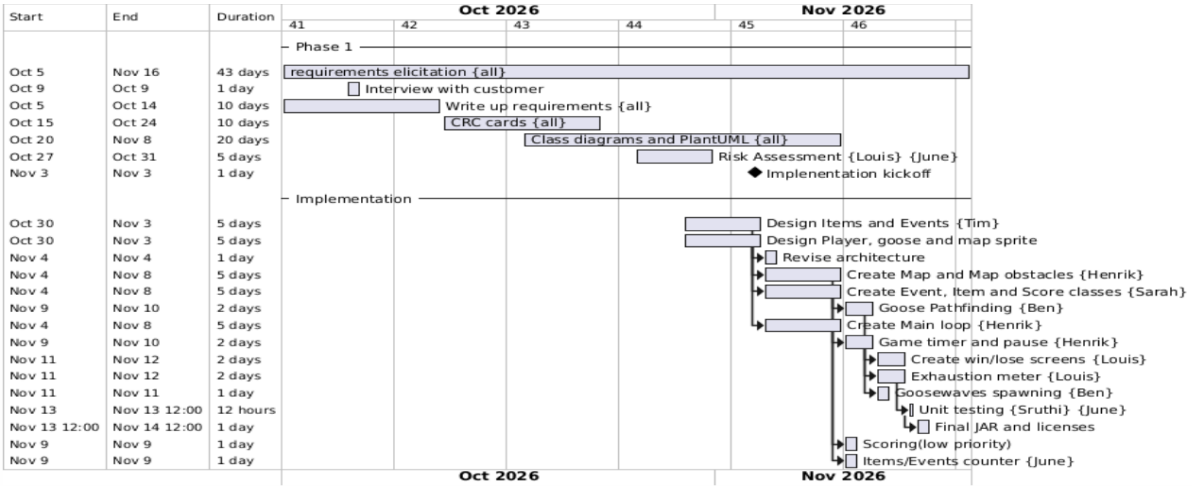


Figure 5: Final Gantt chart

By the final week (9–16 November), our Gantt chart had evolved from a broad outline of requirements, architecture and implementation into a more detailed plan. We had a more detailed breakdown and plan of how to split up tasks and deadlines. This reflected our improved understanding of the game's architecture and allowed us to organise the remaining work before the submission deadline.